Homework #2: Classification Metrics

DATA 621 – Business Analytics and Data Mining

Aaliyah John-Harry | Kevin Havis | Michael Drake | Rebecca Bronstein

October 17, 2026

Overview

This assignment evaluates the performance of a binary classification model using a confusion matrix, accuracy, classification error rate, precision, sensitivity (recall), specificity, F1 score, and an ROC curve with AUC. We first calculate each metric using functions created in R and then compare the results with functions from the `caret` and `pROC` packages.

1. Load the Classification Output Data

```
file_name <- "classification-output-data.csv"

classification_data <- read.csv(file_name)

# Inspect the structure and first few observations.
str(classification_data)
```

```
## 'data.frame':    181 obs. of  11 variables:
##  $ pregnant          : int  7 2 3 1 4 1 9 8 1 2 ...
##  $ glucose           : int  124 122 107 91 83 100 89 120 79 123 ...
##  $ diastolic         : int  70 76 62 64 86 74 62 78 60 48 ...
##  $ skinfold          : int  33 27 13 24 19 12 0 0 42 32 ...
##  $ insulin           : int  215 200 48 0 0 46 0 0 48 165 ...
##  $ bmi               : num  25.5 35.9 22.9 29.2 29.3 19.5 22.5 25 43.5 42.1 ...
##  $ pedigree          : num  0.161 0.483 0.678 0.192 0.317 0.149 0.142 0.409 0.678 0.52 ...
##  $ age               : int  37 26 23 21 34 28 33 64 23 26 ...
##  $ class             : int  0 0 1 0 0 0 0 0 0 0 ...
##  $ scored.class      : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ scored.probability: num  0.328 0.273 0.11 0.056 0.1 ...
```

```
head(classification_data)
```

```
##   pregnant glucose diastolic skinfold insulin  bmi pedigree age class
## 1        7     124        70       33     215 25.5    0.161  37     0
## 2        2     122        76       27     200 35.9    0.483  26     0
## 3        3     107        62       13      48 22.9    0.678  23     1
```

```
## 4        1      91        64       24       0 29.2    0.192  21     0
## 5        4      83        86       19       0 29.3    0.317  34     0
## 6        1     100        74       12      46 19.5    0.149  28     0
##   scored.class scored.probability
## 1            0         0.32845226
## 2            0         0.27319044
## 3            0         0.10966039
## 4            0         0.05599835
## 5            0         0.10049072
## 6            0         0.05515460
```

```
# Confirm the three required columns are available.
required_columns <- c("class", "scored.class", "scored.probability")
stopifnot(all(required_columns %in% names(classification_data)))
```

The three variables used in this assignment are:

- `class`: actual classification
- `scored.class`: predicted classification using a 0.5 threshold
- `scored.probability`: predicted probability of success

```
# Check the distribution of actual classes
class_counts <- table(classification_data$class)

class_proportions <- prop.table(class_counts) * 100

class_distribution <- data.frame(
  Class = names(class_counts),
  Count = as.integer(class_counts),
  Percentage = round(as.numeric(class_proportions), 2)
)

knitr::kable(
  class_distribution,
  caption = "Distribution of Actual Classes"
)
```

Table 1: Distribution of Actual Classes

Class	Count	Percentage
0	124	68.51
1	57	31.49

Checking the distribution of the actual classes, we find that approximately 68.51% of observations are negative (class 0), while 31.49% are positive (class 1).

This gives us a baseline for evaluating the model's performance, since predicting every observation as negative would already result in an accuracy of 68.51%. Keeping this class imbalance in mind will help us better interpret the classification metrics, particularly sensitivity and specificity.

2. Raw Confusion Matrix

```
raw_cm <- table(
  Actual = classification_data$class,
  Predicted = classification_data$scored.class
)

raw_cm
```

```
##        Predicted
## Actual   0   1
##      0 119   5
##      1  30  27
```

Because `class` is supplied as the first argument to `table()` and `scored.class` as the second, the **rows represent the actual classes** and the **columns represent the predicted classes**.

For a binary outcome where `1` is the positive class:

- True Positive (TP): actual = 1 and predicted = 1
- True Negative (TN): actual = 0 and predicted = 0
- False Positive (FP): actual = 0 and predicted = 1
- False Negative (FN): actual = 1 and predicted = 0

Helper Function: Confusion Matrix Counts

The following helper function calculates TP, TN, FP, and FN.

```
confusion_counts <- function(data, actual, predicted, positive = 1) {

  actual_values <- data[[actual]]
  predicted_values <- data[[predicted]]

  negative_values <- setdiff(
    unique(c(actual_values, predicted_values)),
    positive
  )

  if (length(negative_values) != 1) {
    stop("The outcome must be binary and the positive class must be specified correctly.")
  }

  negative <- negative_values[1]

  TP <- sum(actual_values == positive & predicted_values == positive, na.rm = TRUE)
  TN <- sum(actual_values == negative & predicted_values == negative, na.rm = TRUE)
  FP <- sum(actual_values == negative & predicted_values == positive, na.rm = TRUE)
  FN <- sum(actual_values == positive & predicted_values == negative, na.rm = TRUE)
```

```
  c(TP = TP, TN = TN, FP = FP, FN = FN)
}

counts <- confusion_counts(
  classification_data,
  actual = "class",
  predicted = "scored.class",
  positive = 1
)

counts
```

```
##  TP  TN  FP  FN
##  27 119   5  30
```

The confusion matrix compares the actual classifications with the model's predictions. Out of 181 observations, the model correctly classified 119 as class 0 (true negatives) and 27 as class 1 (true positives). However, it incorrectly classified 5 observations as class 1 (false positives) and 30 as class 0 (false negatives).

3. Accuracy

Accuracy is the proportion of all observations that were classified correctly:

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN}$$

```
accuracy <- function(data, actual, predicted, positive = 1) {
  cm <- confusion_counts(data, actual, predicted, positive)

  (cm["TP"] + cm["TN"]) / sum(cm)
}

accuracy_result <- accuracy(
  classification_data,
  "class",
  "scored.class",
  positive = 1
)

cat("accuracy =", (accuracy_result))
```

```
## accuracy = 0.8066298
```

Overall, the model correctly classified 146 out of 181 observations, or approximately **80.66%**.

- While this suggests fairly good overall accuracy, the model performs noticeably better at identifying class 0 than class 1.

- In particular, the relatively high number of false negatives suggests that the model is missing a significant portion of the actual positive cases.

4. Classification Error Rate

The classification error rate is the proportion of observations that were classified incorrectly:

$$Classification\ Error\ Rate = \frac{FP + FN}{TP + FP + TN + FN}$$

```
classification_error <- function(data, actual, predicted, positive = 1) {
  cm <- confusion_counts(data, actual, predicted, positive)

  (cm["FP"] + cm["FN"]) / sum(cm)
}

error_result <- classification_error(
  classification_data,
  "class",
  "scored.class",
  positive = 1
)

cat("Error Rate =", round(unname(error_result), 4), "\n")
```

```
## Error Rate = 0.1934
```

```
# Verify that accuracy + error rate = 1.
cat("Accuracy + Error Rate =",
    round(unname(accuracy_result + error_result), 4), "\n")
```

```
## Accuracy + Error Rate = 1
```

- The classification error rate is approximately **19.34%**, meaning the model incorrectly classified 35 out of 181 observations.
- When combined with the accuracy of 80.66%, the two metrics sum to 100%, confirming that the calculations are correct.

Overall, the model makes correct predictions about 81% of the time and incorrect predictions about 19% of the time.

5. Precision

Precision measures the proportion of predicted positive cases that are actually positive:

$$Precision = \frac{TP}{TP + FP}$$

```r
precision <- function(data, actual, predicted, positive = 1) {
  cm <- confusion_counts(data, actual, predicted, positive)
  denominator <- cm["TP"] + cm["FP"]

  if (denominator == 0) return(NA_real_)

  cm["TP"] / denominator
}

precision_result <- precision(
  classification_data,
  "class",
  "scored.class",
  positive = 1
)

cat("precision =", unname(precision_result), "\n")
```

```
## precision = 0.84375
```

- The model achieved a precision of approximately **84.38%**, meaning that out of the 32 observations predicted as class 1, 27 were actually class 1.
- This suggests that when the model predicts a positive outcome, it is usually correct.
  - However, precision only considers predicted positives and doesn't tell us how many actual positive cases the model failed to identify.

6. Sensitivity (Recall)

Sensitivity (or recall), measures the proportion of actual positive observations correctly identified:

$$Sensitivity = \frac{TP}{TP + FN}$$

```r
sensitivity_custom <- function(data, actual, predicted, positive = 1) {
  cm <- confusion_counts(data, actual, predicted, positive)
  denominator <- cm["TP"] + cm["FN"]

  if (denominator == 0) return(NA_real_)

  cm["TP"] / denominator
}

sensitivity_result <- sensitivity_custom(
  classification_data,
  "class",
  "scored.class",
  positive = 1
```

```
)

cat("sensitivity =", unname(sensitivity_result), "\n")
```

```
## sensitivity = 0.4736842
```

- The model achieved a sensitivity (recall) of approximately **47.37%**, meaning it correctly identified 27 out of 57 actual positive observations.
- This indicates that the model missed more than half of the actual positive cases, incorrectly classifying 30 of them as negative.
  - While the model has relatively high precision (84.38%), its lower sensitivity suggests that it is better at making accurate positive predictions than identifying all positive cases.

7. Specificity

Specificity measures the proportion of actual negative observations correctly identified:

$$Specificity = \frac{TN}{TN + FP}$$

```
specificity_custom <- function(data, actual, predicted, positive = 1) {
  cm <- confusion_counts(data, actual, predicted, positive)
  denominator <- cm["TN"] + cm["FP"]

  if (denominator == 0) return(NA_real_)

  cm["TN"] / denominator
}

specificity_result <- specificity_custom(
  classification_data,
  "class",
  "scored.class",
  positive = 1
)

cat("specificity =", unname(specificity_result), "\n")
```

```
## specificity = 0.9596774
```

- The model achieved a specificity of approximately **95.97%**, meaning it correctly identified 119 out of 124 actual negative observations.
- Only 5 negative observations were incorrectly classified as positive. This suggests that the model performs very well at identifying negative cases.
  - However, compared to its sensitivity of 47.37%, the model is much better at identifying negative observations than positive ones.

8. F1 Score

The F1 score combines precision and sensitivity using their harmonic mean:

$$F1 = \frac{2 \times Precision \times Sensitivity}{Precision + Sensitivity}$$

```
f1_score <- function(data, actual, predicted, positive = 1) {
  p <- precision(data, actual, predicted, positive)
  s <- sensitivity_custom(data, actual, predicted, positive)

  # Return 0 when precision and recall are both zero
  if (!is.na(p) && !is.na(s) && (p + s) == 0) {
    return(0)
  }

  # Handle cases where precision or recall is undefined
  if (is.na(p) || is.na(s)) {
    return(NA_real_)
  }

  2 * p * s / (p + s)
}

f1_result <- f1_score(
  classification_data,
  "class",
  "scored.class",
  positive = 1
)

cat("f1 =", unname(f1_result), "\n")
```

```
## f1 = 0.6067416
```

- The model achieved an F1 score of approximately **60.67%**, which reflects a moderate balance between precision and sensitivity.
- Although the model has relatively high precision (84.38%), its lower sensitivity (47.37%) brings down the overall F1 score. - Suggesting that while the model is usually correct when predicting positive cases, it still misses a significant number of actual positive observations.

9. Bounds of the F1 Score

Precision and sensitivity are both proportions, so their values must fall between 0 and 1. Let a represent precision and b represent sensitivity, where $0 \leq a \leq 1$ and $0 \leq b \leq 1$.

The F1 score is calculated as:

$$F1 = \frac{2ab}{a+b}$$

Since both a and b are nonnegative, the F1 score cannot be negative when $a + b > 0$. Therefore,

$$F1 \geq 0$$

To show that the F1 score cannot exceed 1, we know that:

$$(a-b)^2 \geq 0$$

Expanding this gives:

$$a^2 - 2ab + b^2 \geq 0$$

Rearranging:

$$2ab \leq a^2 + b^2$$

Since a and b are between 0 and 1, we also know that $a^2 \leq a$ and $b^2 \leq b$. Therefore,

$$2ab \leq a^2 + b^2 \leq a + b$$

Dividing by $a + b$, assuming $a + b > 0$, gives:

$$\frac{2ab}{a+b} \leq 1$$

Combining both results, we can conclude that:

$$\boxed{0 \leq F1 \leq 1}$$

This proves that the F1 score is bounded between 0 and 1. A score of 1 occurs when both precision and sensitivity are 1, while a score of 0 occurs when either precision or sensitivity is 0 (provided the denominator is positive). When both are zero, the formula is undefined, but the F1 score is commonly assigned a value of 0 by convention.

10. Custom ROC Curve and AUC Function

An ROC curve evaluates the classifier over many probability thresholds. For each threshold, the predicted class is recalculated and the true positive rate (sensitivity) and false positive rate ($1 - specificity$) are obtained.

The assignment recommends thresholds from 0 to 1 in increments of 0.01.

```r
roc_custom <- function(data,
                       actual,
                       probability,
                       positive = 1,
                       thresholds = seq(0, 1, by = 0.01)) {

  actual_values <- data[[actual]]
  probability_values <- data[[probability]]

  negative_values <- setdiff(unique(actual_values), positive)

  if (length(negative_values) != 1) {
    stop("The actual outcome must be binary
         and the positive class must be specified correctly.")
  }

  negative <- negative_values[1]

  tpr <- numeric(length(thresholds))
  fpr <- numeric(length(thresholds))

  for (i in seq_along(thresholds)) {

    predicted <- ifelse(
      probability_values >= thresholds[i],
      positive,
      negative
    )

    TP <- sum(actual_values == positive & predicted == positive, na.rm = TRUE)
    TN <- sum(actual_values == negative & predicted == negative, na.rm = TRUE)
    FP <- sum(actual_values == negative & predicted == positive, na.rm = TRUE)
    FN <- sum(actual_values == positive & predicted == negative, na.rm = TRUE)

    tpr[i] <- ifelse((TP + FN) == 0, NA, TP / (TP + FN))
    specificity_i <- ifelse((TN + FP) == 0, NA, TN / (TN + FP))
    fpr[i] <- 1 - specificity_i
  }

  roc_points <- data.frame(
    threshold = thresholds,
    FPR = fpr,
    TPR = tpr
  )

  roc_points <- roc_points[complete.cases(roc_points), ]
  roc_points <- roc_points[order(roc_points$threshold, decreasing = TRUE), ]
```

```
  # Trapezoidal approximation of the area under the ROC curve.
  auc <- sum(
    diff(roc_points$FPR) *
      (head(roc_points$TPR, -1) + tail(roc_points$TPR, -1)) / 2
  )

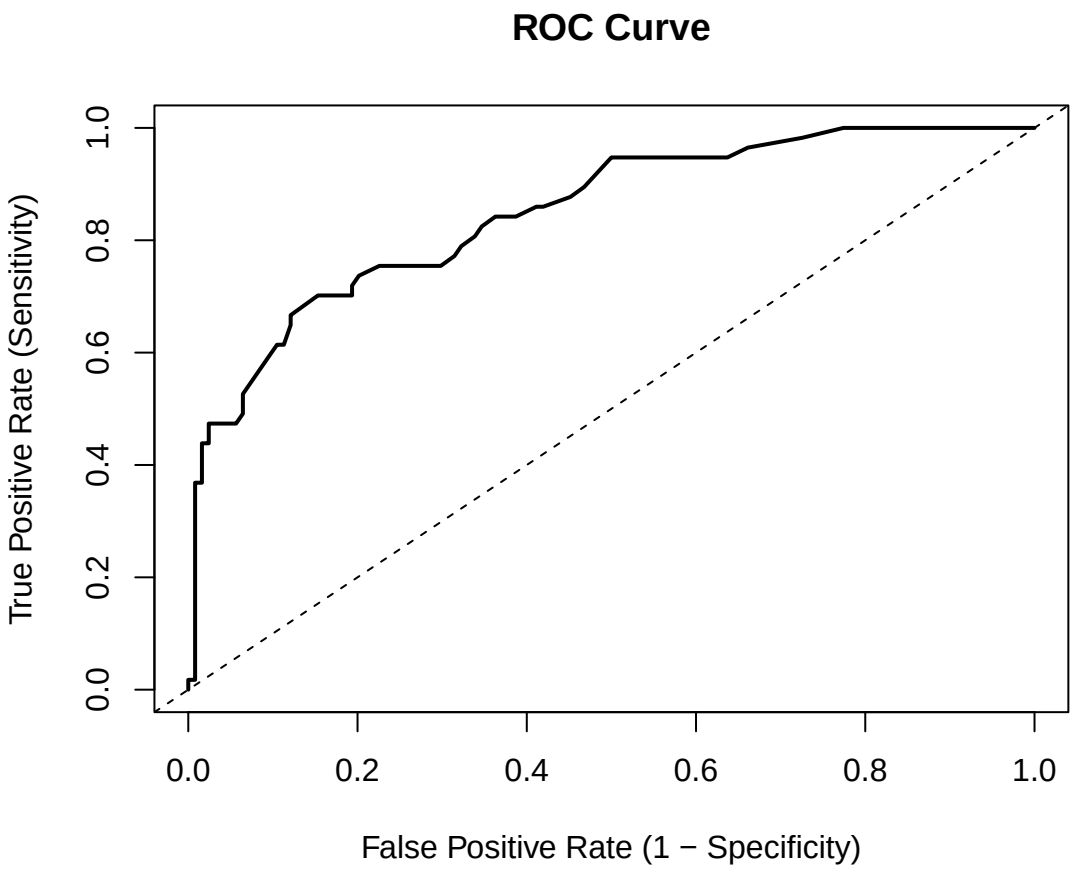
  roc_plot <- plot(
  roc_points$FPR,
  roc_points$TPR,
  type = "l",
  lwd = 2,
  xlim = c(0, 1),
  ylim = c(0, 1),
  xlab = "False Positive Rate (1 - Specificity)",
  ylab = "True Positive Rate (Sensitivity)",
  main = "ROC Curve"
)

abline(a = 0, b = 1, lty = 2)

# Capture the ROC plot
roc_plot <- recordPlot()

return(list(
  plot = roc_plot,
  auc = auc,
  roc_points = roc_points
))
}
```

```
custom_roc <- roc_custom(
  classification_data,
  actual = "class",
  probability = "scored.probability",
  positive = 1
)
```




```
cat("AUC =", custom_roc$auc, "\n")
```

```
## AUC = 0.8488964
```

- The custom ROC curve was generated using classification thresholds ranging from 0 to 1 in increments of 0.01.
- The calculated AUC was approximately **84.89%**, indicating that the model has good ability to distinguish between positive and negative observations. -
- The ROC curve remains above the diagonal reference line, suggesting that the model performs better than random guessing across different classification thresholds.
  - Although the model's sensitivity at the original 0.5 threshold was relatively low (47.37%), the ROC curve shows that adjusting the classification threshold could improve the identification of positive cases, although this may also increase false positives.

11. Classification Metrics Using the Created Functions

The functions created above can now be used together to produce all of the requested classification metrics.

```r
custom_metrics <- data.frame(
  Metric = c(
    "Accuracy",
    "Classification Error Rate",
    "Precision",
    "Sensitivity (Recall)",
    "Specificity",
    "F1 Score",
    "AUC"
  ),
  Value = c(
    accuracy_result,
    error_result,
    precision_result,
    sensitivity_result,
    specificity_result,
    f1_result,
    custom_roc$auc
  )
)

custom_metrics$Value <- round(custom_metrics$Value, 4)

knitr::kable(
  custom_metrics,
  caption = "Classification Metrics from Custom R Functions"
)
```

Table 2: Classification Metrics from Custom R Functions

Metric	Value
Accuracy	0.8066
Classification Error Rate	0.1934
Precision	0.8438
Sensitivity (Recall)	0.4737
Specificity	0.9597
F1 Score	0.6067
AUC	0.8489

The classification metrics show that the model has an overall accuracy of *80.66%*, with an error rate of *19.34%*. While precision (*84.38%*) and specificity (*95.97%*) are relatively high, sensitivity is much lower at *47.37%*, indicating that the model struggles to identify actual positive cases. The F1 score of 0.6067 reflects this imbalance between precision and recall. However, the AUC of *84.89%* suggests that the model has good ability to distinguish between positive and negative observations across different classification thresholds. Overall, the model performs reasonably well, but its ability to identify positive cases could potentially be improved by adjusting the classification threshold.

12. Comparison with the caret Package

```r
library(caret)

actual_factor <- factor(
  classification_data$class,
  levels = c(0, 1)
)

predicted_factor <- factor(
  classification_data$scored.class,
  levels = c(0, 1)
)

# Confusion Matrix
caret_cm <- confusionMatrix(
  data = predicted_factor,
  reference = actual_factor,
  positive = "1"
)

print(caret_cm$table)
```

```
##           Reference
## Prediction   0   1
##          0 119  30
##          1   5  27
```

```r
# Sensitivity
caret_sensitivity <- sensitivity(
  data = predicted_factor,
  reference = actual_factor,
  positive = "1"
)

# Specificity
caret_specificity <- specificity(
  data = predicted_factor,
  reference = actual_factor,
  negative = "0"
)

cat("Sensitivity =", caret_sensitivity, "\n")
```

```
## Sensitivity = 0.4736842
```

```r
cat("Specificity =", caret_specificity, "\n")
```

```
## Specificity = 0.9596774
```

The results from the `caret` package match those calculated using the custom functions. Both approaches produce the same confusion matrix values (TP = 27, TN = 119, FP = 5, FN = 30), along with a sensitivity of 47.37% and specificity of 95.97%.

The model performs much better at identifying negative cases than positive ones, correctly identifying about 96% of negatives but only 47% of positives. The matching results also confirm that the custom functions are working as expected.

Compare the custom results with `caret`:

```
caret_comparison <- data.frame(
  Metric = c("Accuracy", "Sensitivity", "Specificity"),
  Custom = c(
    accuracy_result,
    sensitivity_result,
    specificity_result
  ),
  caret = c(
    unname(caret_cm$overall["Accuracy"]),
    caret_sensitivity,
    caret_specificity
  )
)

caret_comparison$Difference <-
  caret_comparison$Custom - caret_comparison$caret

knitr::kable(
  caret_comparison,
  digits = 6,
  align = c("l", "r", "r", "r"),
  caption = "Custom Functions Compared with caret"
)
```

Table 3: Custom Functions Compared with caret

Metric	Custom	caret	Difference
Accuracy	0.806630	0.806630	0
Sensitivity	0.473684	0.473684	0
Specificity	0.959677	0.959677	0

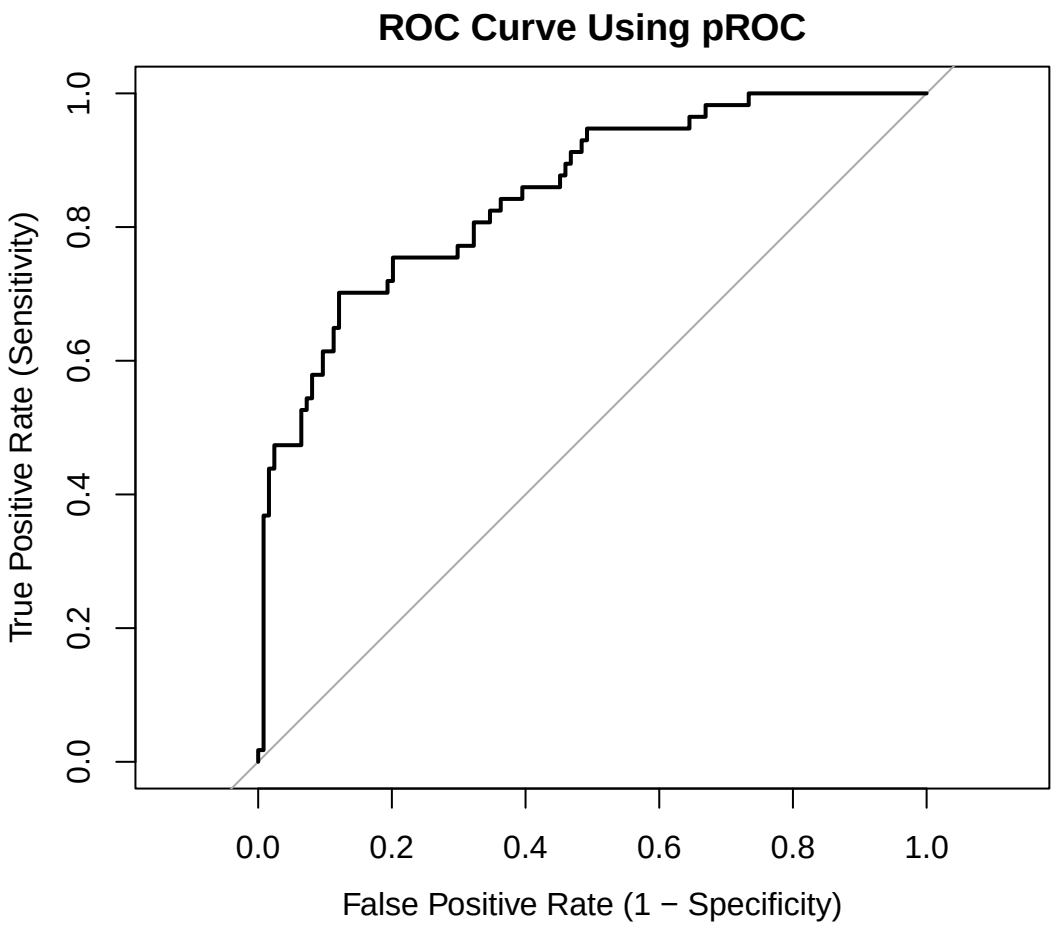
The results from the `caret` package match those calculated using the custom functions, with no differences in accuracy, sensitivity, or specificity. - Both approaches produced an accuracy of 80.66%, sensitivity of 47.37%, and specificity of 95.97%. - The confusion matrix also matches the counts calculated earlier. This confirms that the custom functions are producing the expected results.

13. Comparison with the pROC Package

```r
library(pROC)

proc_roc <- roc(
  response = classification_data$class,
  predictor = classification_data$scored.probability,
  levels = c(0, 1),
  direction = "<",
  quiet = TRUE
)

plot(
  proc_roc,
  main = "ROC Curve Using pROC",
  legacy.axes = TRUE,
  identity = TRUE,
  xlab = "False Positive Rate (1 - Specificity)",
  ylab = "True Positive Rate (Sensitivity)",
  lwd = 2
)
```




```
proc_auc <- as.numeric(auc(proc_roc))
cat("pROC AUC =", proc_auc, "\n")
```

```
## pROC AUC = 0.8503113
```

```
auc_comparison <- data.frame(
  Method = c(
    "Custom ROC function (0.01 threshold increments)",
    "pROC"
  ),
  AUC = c(
    custom_roc$auc,
    proc_auc
  )
)

knitr::kable(
  auc_comparison,
  digits = 4,
  caption = "AUC Comparison"
)
```

Table 4: AUC Comparison

Method	AUC
Custom ROC function (0.01 threshold increments)	0.8489
pROC	0.8503

```
cat("AUC Difference =",
    abs(custom_roc$auc - proc_auc), "\n")
```

```
## AUC Difference = 0.001414827
```

The ROC curves generated by the custom function and the `pROC` package show very similar results. The custom function produced an AUC of 0.8489, while `pROC` returned 0.8503, a difference of approximately 0.0014. This small difference is likely because the custom function uses thresholds in increments of 0.01, while `pROC` evaluates thresholds based on the predicted probabilities. Overall, both methods suggest that the model does a good job distinguishing between positive and negative classifications.

Conclusion

Evaluating the classification model using multiple performance metrics gave us a better understanding of its strengths and weaknesses beyond just overall accuracy. By calculating accuracy, classification error rate, precision, sensitivity, specificity, and F1 score, we were able to see how well the model identified each class and where improvements could be made. Creating our own R functions helped reinforce how these metrics are calculated, while comparing the results with the `caret` package confirmed that our calculations were accurate. Using ROC curves and AUC also allowed us to evaluate the model's ability to distinguish between classes across different classification thresholds, with our custom function producing results nearly identical to those from `pROC`.

Overall, the model achieved an accuracy of 80.66% and an AUC of approximately 0.85, indicating reasonably good classification performance. Considering that 68.51% of the observations belong to the negative class, the model's accuracy represents an improvement over simply predicting the majority class. However, while specificity was high at 95.97%, sensitivity was considerably lower at 47.37%, showing that the model was much better at identifying negative cases than positive ones. This reinforces why accuracy alone is not enough to evaluate classification performance, particularly when the classes are imbalanced. Although the model performs reasonably well overall, we believe improvements could be made by adjusting the classification threshold to better balance sensitivity and specificity, depending on the importance of correctly identifying each class.

Appendix: R Code

This section re-echoes every code chunk above, without re-executing any of it, for reproducibility.

```
knitr::opts_chunk$set(
  echo = TRUE,
  warning = FALSE,
  message = FALSE,
```

```
  fig.align = "center"
)
file_name <- "classification-output-data.csv"

classification_data <- read.csv(file_name)

# Inspect the structure and first few observations.
str(classification_data)
head(classification_data)

# Confirm the three required columns are available.
required_columns <- c("class", "scored.class", "scored.probability")
stopifnot(all(required_columns %in% names(classification_data)))
# Check the distribution of actual classes
class_counts <- table(classification_data$class)

class_proportions <- prop.table(class_counts) * 100

class_distribution <- data.frame(
  Class = names(class_counts),
  Count = as.integer(class_counts),
  Percentage = round(as.numeric(class_proportions), 2)
)

knitr::kable(
  class_distribution,
  caption = "Distribution of Actual Classes"
)
raw_cm <- table(
  Actual = classification_data$class,
  Predicted = classification_data$scored.class
)

raw_cm
confusion_counts <- function(data, actual, predicted, positive = 1) {

  actual_values <- data[[actual]]
  predicted_values <- data[[predicted]]

  negative_values <- setdiff(
    unique(c(actual_values, predicted_values)),
    positive
  )

  if (length(negative_values) != 1) {
    stop("The outcome must be binary and the positive class must be specified correctly.")
  }
```

```
  negative <- negative_values[1]

  TP <- sum(actual_values == positive & predicted_values == positive, na.rm = TRUE)
  TN <- sum(actual_values == negative & predicted_values == negative, na.rm = TRUE)
  FP <- sum(actual_values == negative & predicted_values == positive, na.rm = TRUE)
  FN <- sum(actual_values == positive & predicted_values == negative, na.rm = TRUE)

  c(TP = TP, TN = TN, FP = FP, FN = FN)
}

counts <- confusion_counts(
  classification_data,
  actual = "class",
  predicted = "scored.class",
  positive = 1
)

counts
accuracy <- function(data, actual, predicted, positive = 1) {
  cm <- confusion_counts(data, actual, predicted, positive)

  (cm["TP"] + cm["TN"]) / sum(cm)
}

accuracy_result <- accuracy(
  classification_data,
  "class",
  "scored.class",
  positive = 1
)

cat("accuracy =", (accuracy_result))
classification_error <- function(data, actual, predicted, positive = 1) {
  cm <- confusion_counts(data, actual, predicted, positive)

  (cm["FP"] + cm["FN"]) / sum(cm)
}

error_result <- classification_error(
  classification_data,
  "class",
  "scored.class",
  positive = 1
)

cat("Error Rate =", round(unname(error_result), 4), "\n")
```

```r
# Verify that accuracy + error rate = 1.
cat("Accuracy + Error Rate =",
    round(unname(accuracy_result + error_result), 4), "\n")
precision <- function(data, actual, predicted, positive = 1) {
  cm <- confusion_counts(data, actual, predicted, positive)
  denominator <- cm["TP"] + cm["FP"]

  if (denominator == 0) return(NA_real_)

  cm["TP"] / denominator
}

precision_result <- precision(
  classification_data,
  "class",
  "scored.class",
  positive = 1
)

cat("precision =", unname(precision_result), "\n")
sensitivity_custom <- function(data, actual, predicted, positive = 1) {
  cm <- confusion_counts(data, actual, predicted, positive)
  denominator <- cm["TP"] + cm["FN"]

  if (denominator == 0) return(NA_real_)

  cm["TP"] / denominator
}

sensitivity_result <- sensitivity_custom(
  classification_data,
  "class",
  "scored.class",
  positive = 1
)

cat("sensitivity =", unname(sensitivity_result), "\n")
specificity_custom <- function(data, actual, predicted, positive = 1) {
  cm <- confusion_counts(data, actual, predicted, positive)
  denominator <- cm["TN"] + cm["FP"]

  if (denominator == 0) return(NA_real_)

  cm["TN"] / denominator
}

specificity_result <- specificity_custom(
```

```
  classification_data,
  "class",
  "scored.class",
  positive = 1
)

cat("specificity =", unname(specificity_result), "\n")
f1_score <- function(data, actual, predicted, positive = 1) {
  p <- precision(data, actual, predicted, positive)
  s <- sensitivity_custom(data, actual, predicted, positive)

  # Return 0 when precision and recall are both zero
  if (!is.na(p) && !is.na(s) && (p + s) == 0) {
    return(0)
  }

  # Handle cases where precision or recall is undefined
  if (is.na(p) || is.na(s)) {
    return(NA_real_)
  }

  2 * p * s / (p + s)
}

f1_result <- f1_score(
  classification_data,
  "class",
  "scored.class",
  positive = 1
)

cat("f1 =", unname(f1_result), "\n")
roc_custom <- function(data,
                       actual,
                       probability,
                       positive = 1,
                       thresholds = seq(0, 1, by = 0.01)) {

  actual_values <- data[[actual]]
  probability_values <- data[[probability]]

  negative_values <- setdiff(unique(actual_values), positive)

  if (length(negative_values) != 1) {
    stop("The actual outcome must be binary
         and the positive class must be specified correctly.")
  }
```

```r
negative <- negative_values[1]

tpr <- numeric(length(thresholds))
fpr <- numeric(length(thresholds))

for (i in seq_along(thresholds)) {

  predicted <- ifelse(
    probability_values >= thresholds[i],
    positive,
    negative
  )

  TP <- sum(actual_values == positive & predicted == positive, na.rm = TRUE)
  TN <- sum(actual_values == negative & predicted == negative, na.rm = TRUE)
  FP <- sum(actual_values == negative & predicted == positive, na.rm = TRUE)
  FN <- sum(actual_values == positive & predicted == negative, na.rm = TRUE)

  tpr[i] <- ifelse((TP + FN) == 0, NA, TP / (TP + FN))
  specificity_i <- ifelse((TN + FP) == 0, NA, TN / (TN + FP))
  fpr[i] <- 1 - specificity_i
}

roc_points <- data.frame(
  threshold = thresholds,
  FPR = fpr,
  TPR = tpr
)

roc_points <- roc_points[complete.cases(roc_points), ]
roc_points <- roc_points[order(roc_points$threshold, decreasing = TRUE), ]

# Trapezoidal approximation of the area under the ROC curve.
auc <- sum(
  diff(roc_points$FPR) *
    (head(roc_points$TPR, -1) + tail(roc_points$TPR, -1)) / 2
)

roc_plot <- plot(
roc_points$FPR,
roc_points$TPR,
type = "l",
lwd = 2,
xlim = c(0, 1),
ylim = c(0, 1),
xlab = "False Positive Rate (1 - Specificity)",
ylab = "True Positive Rate (Sensitivity)",
```

```
  main = "ROC Curve"
)

abline(a = 0, b = 1, lty = 2)

# Capture the ROC plot
roc_plot <- recordPlot()

return(list(
  plot = roc_plot,
  auc = auc,
  roc_points = roc_points
))
}
custom_roc <- roc_custom(
  classification_data,
  actual = "class",
  probability = "scored.probability",
  positive = 1
)

cat("AUC =", custom_roc$auc, "\n")

custom_metrics <- data.frame(
  Metric = c(
    "Accuracy",
    "Classification Error Rate",
    "Precision",
    "Sensitivity (Recall)",
    "Specificity",
    "F1 Score",
    "AUC"
  ),
  Value = c(
    accuracy_result,
    error_result,
    precision_result,
    sensitivity_result,
    specificity_result,
    f1_result,
    custom_roc$auc
  )
)

custom_metrics$Value <- round(custom_metrics$Value, 4)

knitr::kable(
```

```
  custom_metrics,
  caption = "Classification Metrics from Custom R Functions"
)


library(caret)

actual_factor <- factor(
  classification_data$class,
  levels = c(0, 1)
)

predicted_factor <- factor(
  classification_data$scored.class,
  levels = c(0, 1)
)

# Confusion Matrix
caret_cm <- confusionMatrix(
  data = predicted_factor,
  reference = actual_factor,
  positive = "1"
)

print(caret_cm$table)

# Sensitivity
caret_sensitivity <- sensitivity(
  data = predicted_factor,
  reference = actual_factor,
  positive = "1"
)

# Specificity
caret_specificity <- specificity(
  data = predicted_factor,
  reference = actual_factor,
  negative = "0"
)

cat("Sensitivity =", caret_sensitivity, "\n")
cat("Specificity =", caret_specificity, "\n")
caret_comparison <- data.frame(
  Metric = c("Accuracy", "Sensitivity", "Specificity"),
  Custom = c(
    accuracy_result,
    sensitivity_result,
```

```
    specificity_result
  ),
  caret = c(
    unname(caret_cm$overall["Accuracy"]),
    caret_sensitivity,
    caret_specificity
  )
)

caret_comparison$Difference <-
  caret_comparison$Custom - caret_comparison$caret

knitr::kable(
  caret_comparison,
  digits = 6,
  align = c("l", "r", "r", "r"),
  caption = "Custom Functions Compared with caret"
)

library(pROC)

proc_roc <- roc(
  response = classification_data$class,
  predictor = classification_data$scored.probability,
  levels = c(0, 1),
  direction = "<",
  quiet = TRUE
)

plot(
  proc_roc,
  main = "ROC Curve Using pROC",
  legacy.axes = TRUE,
  identity = TRUE,
  xlab = "False Positive Rate (1 - Specificity)",
  ylab = "True Positive Rate (Sensitivity)",
  lwd = 2
)

proc_auc <- as.numeric(auc(proc_roc))
cat("pROC AUC =", proc_auc, "\n")

auc_comparison <- data.frame(
  Method = c(
    "Custom ROC function (0.01 threshold increments)",
    "pROC"
  ),
```

```
  AUC = c(
    custom_roc$auc,
    proc_auc
  )
)

knitr::kable(
  auc_comparison,
  digits = 4,
  caption = "AUC Comparison"
)

cat("AUC Difference =",
    abs(custom_roc$auc - proc_auc), "\n")
```